

PLAN.md — แผนการพัฒนาโครงการ Wikanda Hair Salon

เอกสารนี้คือ **Single Source of Truth** สำหรับการพัฒนาโครงการ AI หรือนักพัฒนาคนใหม่ที่เข้ามาต้องงาน ให้อ่านไฟล์นี้ก่อนเป็นอันดับแรก อัปเดต Checklist ทุกครั้งที่ทำงานเสร็จ

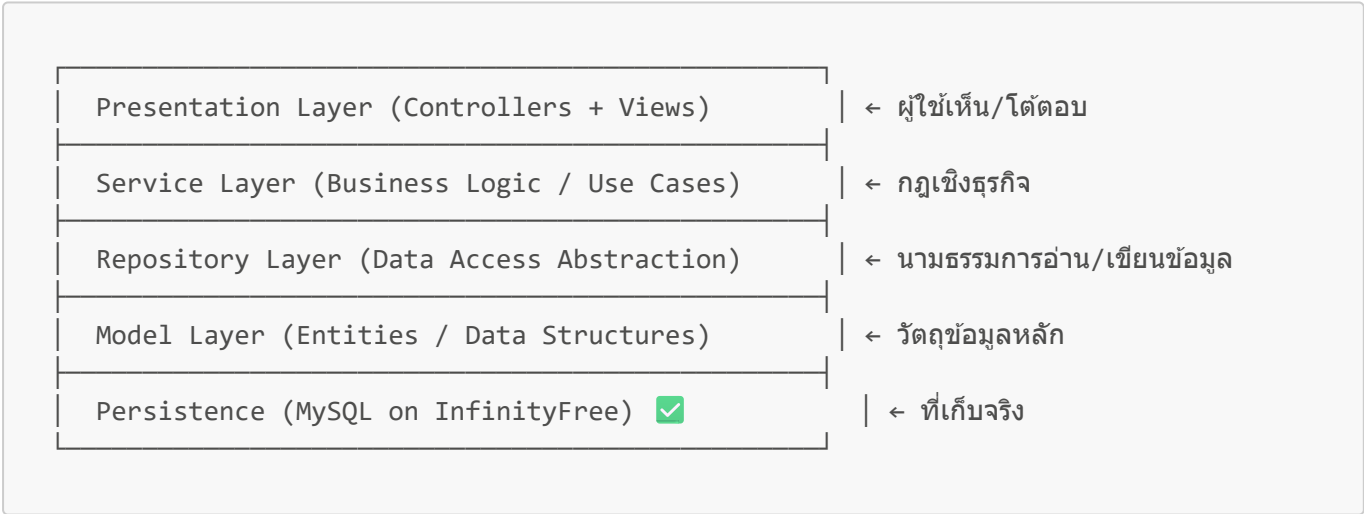
1. เป้าหมายโครงการ (Project Goal)

พัฒนา Web Application สำหรับบริหารจัดการร้านเสริมสวย Wikanda Hair Salon ครอบคลุม: ระบบจองคิว, ชำระเงิน, แจ้งเตือน LINE OA, รายงาน Dashboard

Tech Stack:

- Backend: **PHP 8+** (Pure PHP, ไม่ใช้ Framework)
- Frontend: **HTML + CSS + JavaScript + Bootstrap 5**
- Database: **MySQL** (InfinityFree Hosting) ☒ *Migrated from JSON*
- Notification: **LINE Messaging API**
- Slip Verification: **Slip2Go API** (<https://slip2go.com>)
- Local Server: **XAMPP** (Apache + MySQL + PHP) — ห้ามใช้ Docker

2. สถาปัตยกรรม Clean Architecture



หลักการสำคัญ:

- ☒ Layer บนเรียก Layer ล่างได้ — Layer ล่างห้ามรู้จัก Layer บน
- ☒ Controller บางลง (Thin Controller) — ตรรกะอยู่ใน Service
- ☒ Repository ปิดบังว่าข้อมูลมาจาก JSON หรือ MySQL — สลับได้โดยไม่กระทบ Service
- ☒ Model = โครงสร้างข้อมูลล้วน ไม่มี Logic ฐานข้อมูล

3. โครงสร้างไฟล์ (Folder Structure)

Wikanda_Hair_Salon/	
├── PLAN.md	← เอกสารนี้
├── PROJECT_ANALYSIS.md	← การวิเคราะห์ข้อเสนอ
├── README.md	← วิธีติดตั้ง/ใช้งาน
├── api/	← REST API (คืนค่าเป็น JSON)
│ ├── index.php	← API Router + CORS + Error Handler
│ └── v1/	← API Version 1
│ ├── auth.php	← POST /api/v1/auth/login register logout
│ ├── users.php	← CRUD users
│ ├── services.php	← CRUD services
│ ├── staff.php	← CRUD staff
│ ├── bookings.php	← CRUD bookings + status update
│ ├── payments.php	← CRUD payments + verify slip
│ └── reviews.php	← CRUD reviews
├── public/	← Web Root (Apache ขึ้นมาที่นี่)
│ ├── index.php	← Front Controller (Web UI)
│ ├── .htaccess	← URL Rewrite
│ └── assets/	
│ ├── css/style.css	← Custom CSS (Lovable Style)
│ ├── js/main.js	← Vanilla JS เรียก /api/v1/*
│ └── images/	
├── app/	
│ ├── Core/	← Framework Core (ทำเองแบบเบา)
│ │ ├── Router.php	← จับ URL → เรียก Controller
│ │ ├── Controller.php	← Base Controller
│ │ ├── Database.php	← จัดการ JSON Files (Legacy)
│ │ ├── MysqlDatabase.php	← จัดการ MySQL Database ✓
│ │ ├── DatabaseInterface.php	← Interface สำหรับ Database
│ │ ├── Session.php	← Session Helper
│ │ ├── Request.php	← ห่อ \$_GET/\$_POST
│ │ └── View.php	← Render Template
│ ├── Models/	← Entity (โครงสร้างข้อมูล)
│ │ ├── User.php	
│ │ ├── Service.php	← ประเภทบริการ (ตัด/ย้อม/ตัด)
│ │ ├── Staff.php	
│ │ ├── Booking.php	
│ │ ├── Payment.php	
│ │ └── Review.php	
│ ├── Repositories/	← Data Access Layer
│ │ ├── BaseRepository.php	← CRUD พื้นฐาน (รองรับ JSON/MySQL)
│ │ ├── UserRepository.php	
│ │ ├── ServiceRepository.php	
│ │ ├── StaffRepository.php	
│ │ ├── BookingRepository.php	
│ │ ├── PaymentRepository.php	
│ │ ├── ReviewRepository.php	
│ │ └── SettingRepository.php	

Services/	← Business Logic
— AuthService.php	← Login/Register/Logout
— BookingService.php	← จองคิว / ตรวจคิวซ้อน
— PaymentService.php	← จัดการชำระเงิน
— ReportService.php	← สรุปรายงาน Dashboard
— LineNotifyService.php	← ส่งข้อความ LINE OA
Controllers/	← HTTP Request Handlers
— HomeController.php	
— AuthController.php	
— MemberController.php	← หน้าสมาชิก
— BookingController.php	← จองคิว
— StaffController.php	← หน้าพนักงาน
— AdminController.php	← หน้าผู้ดูแล
Middleware/	← ตัวกรองคำขอ
— AuthMiddleware.php	← เช็คล็อกอิน + Role
Views/	← HTML Templates
— layouts/	
— main.php	← Layout หลัก
— partials/	
— navbar.php	
— footer.php	
— home/	
— auth/	
— member/	
— staff/	
— admin/	
config/	
— app.php	← ตั้งค่าทั่วไป
— routes.php	← นิยาม URL → Controller
— line.php	← LINE OA Token (อย่า commit จริง)
— slip2go.php	← Slip2Go API Token (อย่า commit จริง)
data/	← JSON "ฐานข้อมูล"
— users.json	
— services.json	
— staff.json	
— bookings.json	
— payments.json	
— reviews.json	
storage/	
— uploads/	
— slips/	← เก็บสลิปการชำระเงิน

4. กฎเหล็กสำหรับการเขียนโค้ด (Coding Rules)

4.1 Comment ทุกไฟล์/ฟังก์ชัน ภาษาไทย-อังกฤษ

```
/**
 * จองคิวให้ลูกค้า / Create a new booking for the customer
 *
 * @param int $memberId รหัสสมาชิก / Member ID
 * @param int $serviceId รหัสบริการ / Service ID
 * @return array ข้อมูลการจอง / Booking data
 */
```

4.2 ตั้งชื่อตัวแปร/ฟังก์ชันแบบสื่อความหมาย

- ✗ \$d, \$x, \$tmp
- ✓ \$bookingDate, \$memberId, \$totalRevenue

4.3 เขียนสั้น เข้าใจง่าย ไม่อวดเทคนิค

- 1 ฟังก์ชัน = 1 หน้าที่
- หลีกเลี่ยง Magic, Trick, One-liner ชับซ้อน
- เด็กจบใหม่อ่านแล้วเข้าใจภายใน 30 วินาที

4.4 ใช้ Type Hint และ Return Type เสมอ

```
public function findById(int $id): ?array
```

4.5 ห้ามทำใน Controller

- ✗ เปิดไฟล์ JSON ตรง ๆ → ใช้ Repository
- ✗ คำนวณราคา/ตรวจคิวซ้อน → ใช้ Service
- ✗ Echo HTML ออกมา → ใช้ View

4.6 ทุก Table ต้องมี UUID + Auto Increment ID

ทุกแถวในทุก JSON ต้องมี **2 key** หลัก:

- id** — integer auto-increment (สำหรับ FK + ตรงกับ MySQL PRIMARY KEY)
- uuid** — UUID v4 string สำหรับใช้ในระดับ public/URL (ปลอดภัยกว่า expose id ตรง ๆ)

ตัวอย่าง:

```
{
  "id": 1,
  "uuid": "0c8a4f6e-9d2b-4c1f-8e7a-1b2c3d4e5f6a",
  "username": "admin",
  ...
}
```

Helper สำหรับสร้าง UUID อยู่ที่ `app/Core/Database.php::generateUuid()`

✅ 5. Checklist การพัฒนา (ทำตามลำดับ)

Phase 1: รากฐาน (Foundation) 🏗️

- ✅ วิเคราะห์ข้อเสนอโครงการ (PROJECT_ANALYSIS.md)
- ✅ สร้าง PLAN.md (ไฟล์นี้)
- ✅ สร้างโครงสร้างโฟลเดอร์ทั้งหมด
- ✅ สร้างไฟล์ JSON พร้อมข้อมูลตัวอย่าง (sample data) + UUID
 - ✅ users.json (Admin, Owner, Staff x2, Member x2)
 - ✅ services.json (7 บริการ)
 - ✅ staff.json (2 ช่าง)
 - ✅ bookings.json (sample 5 รายการ)
 - ✅ payments.json (3 รายการ)
 - ✅ reviews.json (2 รายการ)
- ✅ เขียนไฟล์ config (`app.php`, `routes.php`, `line.php`, `slip2go.php`)

Phase 2: Core Framework 🛠️

- ✅ `Core/Database.php` — อ่าน/เขียน JSON + UUID generator
- ✅ `Core/Session.php` — Session helper
- ✅ `Core/Request.php` — ห่อ Input + JSON body parser
- ✅ `Core/Router.php` — URL → Controller (รองรับ {param})
- ✅ `Core/Controller.php` — Base Controller
- ✅ `Core/View.php` — Render Template
- ✅ `Core/autoload.php` — PSR-4 autoloader ⚠️ สำคัญ ต้องทำ
- ✅ `public/index.php` — Front Controller
- ✅ `public/.htaccess` — URL Rewrite

Phase 3: Domain Layer (Models + Repositories) 📦

- ✅ Models ทั้ง 6 ตัว (User, Service, Staff, Booking, Payment, Review)
- ✅ `Repositories/BaseRepository.php` — CRUD พื้นฐาน
- ✅ Repository ทั้ง 6 ตัว (extends BaseRepository)

Phase 4: Business Logic (Services) 💬

- ✅ `AuthService` — Register / Login / Logout / Hash Password
- ✅ `BookingService` — จองคิว, ตรวจสอบเวลาซ้อน, ยกเลิก
- ✅ `PaymentService` — สร้างใบชำระ, แนนสลิป, อนุมัติ
- ✅ `ReportService` — สรุปรายได้ราย วัน/เดือน/ปี
- ✅ `LineNotifyService` — ส่งข้อความ LINE (Stub ก่อน, ค่อยต่อ API จริง)
- ✅ `Slip2GoService` — เรียก Slip2Go API ตรวจสอบสลิปอัตโนมัติ

Phase 5a: REST API Layer (`/api/v1/*.php`) 🌐

- ✅ `api/index.php` — API Router + CORS + Global Error Handler + JSON Response Helper

- ☒ `api/v1/auth.php` — login, register, logout
- ☒ `api/v1/services.php` — CRUD services
- ☒ `api/v1/staff.php` — CRUD staff (read-only public, write admin)
- ☒ `api/v1/bookings.php` — CRUD + status update
- ☒ `api/v1/payments.php` — CRUD + verify slip
- ☒ `api/v1/reviews.php` — CRUD reviews
- ☒ `api/v1/users.php` — จัดการ users (admin only)

Phase 5b: Presentation Layer (Controllers + Views) 🎨

- ☒ `Middleware/AuthMiddleware.php`
- ☒ Base Layout (`Views/layouts/main.php`) + Bootstrap 5
- ☒ Navbar / Footer Partials
- ☒ **HomeController** — หน้าแรก (ดูบริการ + โปรโมชัน)
- ☒ **AuthController** — Login / Register / Logout
- ☒ **MemberController** — Dashboard สมาชิก, ประวัติการจอง
- ☒ **BookingController** — เลือกบริการ, ช่วง, เวลา, แบนสลิป
- ☒ **StaffController** — ดูคิวงานวันนี้, อัปเดตสถานะ
- ☒ **AdminController** — จัดการ Master Data, อนุมัติชำระเงิน
- ☒ Owner Dashboard (Reuse AdminController + เพิ่มหน้า Report)

Phase 6: Integration & Polish 🚀

- ☒ CSS Style ปรับให้สวย — **สไตล์ Lovable** (modern, minimalist, soft gradient, glassmorphism, large rounded corners 16-24px, soft shadows, smooth animations, Inter/Plus Jakarta Sans font, generous whitespace, card-based)
 - Palette แนะนำ: ชมพูพาสเทล (#FFE0EC), ม่วงอ่อน (#E8DAFF), ครีม (#FFF8F3), accent: gradient ชมพู→ม่วง
- ☒ JavaScript: Validation ฟอर्म, Date Picker, Confirm Dialog
- ☐ LINE OA: ต่อ API จริง + ทดสอบส่งข้อความ
- ☒ ทดสอบทุก User Role (5 บทบาท)
- ☒ เขียน README.md (วิธี install / run / ทดสอบ)

Phase 7: Migration to MySQL (อนาคต) 🗄️

- ☒ ออกแบบ ER Diagram
- ☒ สร้าง MySQL Schema
- ☐ เขียน Repository ใหม่เป็น `MysqlUserRepository` etc.
- ☐ เปลี่ยน Service Container ให้ใช้ MySQL Repo
- ☐ เขียน Migration Script ย้ายข้อมูลจาก JSON → MySQL

🔑 6. ข้อมูลทดสอบเริ่มต้น (Test Credentials)

สร้างไว้ใน `data/users.json` พร้อมรหัสผ่าน hash แล้ว รหัสผ่าน plain text ทุกบัญชี: **password123**

Role	Username	Email
Admin	admin	admin@wikanda.local

Role	Username	Email
Owner	owner	owner@wikanda.local
Staff	staff01	staff01@wikanda.local
Staff	staff02	staff02@wikanda.local
Member	member01	member01@example.com

7. URL Routes (วางแผนล่องหน้า)

7.1 Web Routes (HTML Pages)

Method	URL	Controller@Action	Role
GET	/	Home@index	Public
GET	/services	Home@services	Public
GET	/login	Auth@showLogin	Public
GET	/register	Auth@showRegister	Public
GET	/member	Member@dashboard	Member
GET	/booking/new	Booking@create	Member
GET	/booking/{id}	Booking@show	Member
GET	/staff	Staff@dashboard	Staff
GET	/admin	Admin@dashboard	Admin/Owner
GET	/admin/services	Admin@services	Admin/Owner
GET	/admin/bookings	Admin@bookings	Admin/Owner
GET	/admin/report	Admin@report	Owner

7.2 REST API Routes (/api/v1/*) — คีน JSON

Method	URL	Description	Role
POST	/api/v1/auth/register	สมัครสมาชิก	Public
POST	/api/v1/auth/login	เข้าสู่ระบบ	Public
POST	/api/v1/auth/logout	ออกจากระบบ	Logged In
GET	/api/v1/services	รายการบริการทั้งหมด	Public
GET	/api/v1/services/{id}	บริการรายตัว	Public
POST	/api/v1/services	เพิ่มบริการ	Admin/Owner
PUT	/api/v1/services/{id}	แก้ไขบริการ	Admin/Owner

Method	URL	Description	Role
DELETE	/api/v1/services/{id}	ลบบริการ	Admin/Owner
GET	/api/v1/staff	รายการช่าง	Public
GET	/api/v1/bookings	รายการจอง (ตาม role)	Logged In
POST	/api/v1/bookings	สร้างการจอง	Member
PUT	/api/v1/bookings/{id}/status	อัปเดตสถานะ	Staff/Admin
DELETE	/api/v1/bookings/{id}	ยกเลิกการจอง	Member/Admin
POST	/api/v1/payments	สร้างใบชำระ + แนนสลิป	Member
POST	/api/v1/payments/{id}/verify	ตรวจสอบสลิปผ่าน Slip2Go	Admin
POST	/api/v1/reviews	รีวิวบริการ	Member

รูปแบบ Response มาตรฐาน:

```
{
  "success": true,
  "message": "OK",
  "data": { ... }
}
```

```
{
  "success": false,
  "message": "Validation failed",
  "errors": { "field": ["..."] }
}
```

 8. คำสั่งสำคัญ (Cheatsheet สำหรับ AI/Dev ต่อ)

เริ่มต้นใหม่จากศูนย์

1. อ่าน PLAN.md (ไฟล์นี้)
 2. ดู PROJECT_ANALYSIS.md สำหรับ context
 3. ดูสถานะ Checklist ในข้อ 5
 4. ทำต่อจากข้อที่ยังเป็น []

กฎการอัปเดต PLAN.md

-  ทำงานเสร็จ → เปลี่ยน - [] เป็น - [x] ทันที
-  เจอ Task ใหม่ → เพิ่มในส่วนที่เหมาะสม

- ✅ เปลี่ยนแผน → แก้ไขส่วนที่เกี่ยวข้อง + บันทึกเหตุผลใน Section 9

 9. บันทึกการเปลี่ยนแปลง (Change Log)

วันที่	ผู้ทำ	รายละเอียด
2026-05-18	Claude (Initial)	สร้าง PLAN.md และเริ่ม Phase 1
2026-05-18	User Clarification	ยืนยันใช้ XAMPP (ไม่ใช่ Docker) และตรวจสอบสลิปด้วย Slip2Go API
2026-05-18	User Direction	UI ต้องเป็นสไตล์ Lovable — modern, minimalist, soft gradient, glassmorphism
2026-05-18	User Direction	เพิ่มโฟลเดอร์ <code>/api</code> เป็น REST API (PHP) ใช้ JSON เป็น storage ก่อน
2026-05-18	User Direction	ทุกตารางต้องมีคอลัมน์ <code>uuid</code> (UUID v4) เพิ่มจาก <code>id</code>

 10. ข้อควรระวัง (Warnings)

- อย่า **commit** `config/line.php` ที่มี Access Token จจริงลง Git
- อย่าเก็บ **Password** แบบ **plain text** — ใช้ `password_hash()` เสมอ
- JSON** ไม่มี **Transaction** — ระวัง Race Condition (ใช้ `flock()` ใน Database.php)
- อย่าใส่ **Logic** ใน **View** — Logic อยู่ใน Service, View แสดงผลอย่างเดียว
- `storage/uploads/slips/` ต้องมี **permission write** — แต่ห้ามให้ web เข้าถึงเป็น public โดยตรง

11. Current Completion Update - 2026-06-06

Updated after reading all project markdown files: `PLAN.md`, `PROJECT_ANALYSIS.md`, `PROJECT_OVERVIEW.md`, `README.md`, `USER_GUIDE.md`, and `HANDOFF.md`.

Completed / verified:

- Phase 1 Foundation: folder structure, JSON sample data, config files.
- Phase 2 Core Framework: autoload, request/session/router/controller/view/database, front controller, `.htaccess`.
- Phase 3 Domain Layer: all 6 models and repositories.
- Phase 4 Business Logic: auth, booking, payment, report, LINE service wrapper, Slip2Go service wrapper.
- Phase 5a REST API: API router and all v1 handlers for auth, services, staff, bookings, payments, reviews, reports, and users.
- Phase 5b Presentation: middleware, layout, navbar/footer/sidebar, home/auth/member/booking/staff/admin pages.
- Phase 6 Polish: premium responsive UI, Thai font stack, modern page layouts, form validation, date guard, booking time validation, confirm helper, API helper, sidebar state.

- Documentation added from project analysis gaps: `docs/ER_DIAGRAM.md`, `docs/USE_CASE_DIAGRAM.md`, `docs/ACTIVITY_FLOW.md`, `docs/SYSTEM_ARCHITECTURE.md`, `docs/WIREFRAME.md`, `docs/TEST_PLAN.md`.

Verified:

- `php -l` passes for every PHP file in the project.
- Public UI loads at `http://127.0.0.1:8087/Wikanda_Hair_Salon/public/?v=ui2`.
- CSS and JS load from the correct local host.
- Public page has no horizontal overflow in the checked viewport.
- `/api/v1/services` returns JSON successfully.
- `/api/v1/auth` login succeeds for the admin test account.
- Admin session cookie can access `/public/admin`.
- Unauthenticated `/public/admin` redirects to `/public/login` on the current browser host.

Still external/config dependent:

- Real LINE Messaging API send test requires a real LINE channel token in `config/line.php`.
- Real Slip2Go verification requires a real token in `config/slip2go.php`.
- Phase 7 MySQL migration is intentionally future scope per project plan.